



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Approved by AICTE, Affiliated to JNTUH, Hyderabad-500029

Prepared By:

M. Asha Jyothi

Assistant Professor, CSE(AIML)

KMIT, Hyderabad

EXERCISE 3 — VARIATIONAL AUTOENCODER

Problem Statement

VAE-Based Fashion Image Reconstruction and Generation

Build a Variational Autoencoder using the Fashion-MNIST dataset.

Unlike a conventional autoencoder, the VAE should learn a probability distribution in the latent space rather than mapping each input to a single fixed latent vector.

The model should: **bold text** encode Fashion-MNIST images into a latent distribution

sample latent vectors from the learned distribution

reconstruct the input image

visualize the latent representation

generate new Fashion-MNIST-like images from randomly sampled latent vectors

Objectives

Students should understand:

difference between AE and VAE

latent vector vs latent distribution

mean μ

variance / log variance

reparameterization trick

reconstruction loss

KL divergence

total VAE loss

how VAE can generate new samples

AE learns a compressed representation; VAE learns a structured probability distribution from which representations can be sampled.

```
# =====
# EXERCISE 3: VARIATIONAL AUTOENCODER (VAE)
# Dataset: Fashion-MNIST
# =====

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras import layers, Model

# =====
# 1. LOAD DATASET
# =====

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

print("Original Training Shape:", x_train.shape)
print("Original Testing Shape :", x_test.shape)

# =====
# 2. NORMALIZE
# =====

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# =====
# 3. ADD CHANNEL DIMENSION
# 28 x 28 --> 28 x 28 x 1
# =====

x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)

# =====
```

```

# 4. VAE PARAMETERS
# =====

latent_dim = 2
input_shape = (28, 28, 1)

# =====
# 5. ENCODER
# =====

encoder_inputs = layers.Input(
    shape=input_shape,
    name="encoder_input"
)

x = layers.Flatten()(encoder_inputs)

x = layers.Dense(
    256,
    activation="relu"
)(x)

x = layers.Dense(
    128,
    activation="relu"
)(x)

# -----
# Mean of latent distribution
# -----

z_mean = layers.Dense(
    latent_dim,
    name="z_mean"
)(x)

# -----
# Log variance of latent distribution
# -----

z_log_var = layers.Dense(
    latent_dim,
    name="z_log_var"
)(x)

# =====

```

```
# 6. REPARAMETERIZATION / SAMPLING
```

```
# =====
```

```
class Sampling(layers.Layer):
```

```
    def call(self, inputs):
```

```
        z_mean, z_log_var = inputs
```

```
        # Random noise
```

```
        epsilon = tf.random.normal(  
            shape=tf.shape(z_mean)
```

```
        )
```

```
        # Reparameterization trick
```

```
        z = (  
            z_mean  
            + tf.exp(0.5 * z_log_var)  
            * epsilon  
        )
```

```
        return z
```

```
z = Sampling(name="sampling")(  
    [z_mean, z_log_var]  
)
```

```
# =====
```

```
# 7. CREATE ENCODER MODEL
```

```
# =====
```

```
encoder = Model(  
    encoder_inputs,  
    [z_mean, z_log_var, z],  
    name="encoder"  
)
```

```
print("\n=====")
```

```
print("ENCODER SUMMARY")
```

```
print("=====")
```

```
encoder.summary()
```

```
# =====
```

```
# 8. DECODER
```

```
# =====
```

```
latent_inputs = layers.Input(
```

```

        shape=(latent_dim,),
        name="decoder_input"
    )

x = layers.Dense(
    128,
    activation="relu"
)(latent_inputs)

x = layers.Dense(
    256,
    activation="relu"
)(x)

x = layers.Dense(
    28 * 28,
    activation="sigmoid"
)(x)

decoder_outputs = layers.Reshape(
    (28, 28, 1)
)(x)

# =====
# 9. CREATE DECODER MODEL
# =====

decoder = Model(
    latent_inputs,
    decoder_outputs,
    name="decoder"
)

print("\n=====")
print("DECODER SUMMARY")
print("=====")

decoder.summary()

# =====
# 10. DEFINE VAE
# =====

class VAE(Model):
    def __init__(
        self,
        encoder,

```

```

        decoder,
        **kwargs
    ):
        super().__init__(**kwargs)

        self.encoder = encoder
        self.decoder = decoder

        # -----
        # Metrics
        # -----

        self.total_loss_tracker = tf.keras.metrics.Mean(
            name="total_loss"
        )

        self.reconstruction_loss_tracker = tf.keras.metrics.Mean(
            name="reconstruction_loss"
        )

        self.kl_loss_tracker = tf.keras.metrics.Mean(
            name="kl_loss"
        )

        # =====
        # METRICS
        # =====

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker
        ]

        # =====
        # CALL METHOD
        # Needed by Keras for validation/prediction
        # =====

    def call(
        self,
        inputs,
        training=False
    ):

```

```

    # Encoder
    z_mean, z_log_var, z = self.encoder(
        inputs,
        training=training
    )

    # Decoder
    reconstruction = self.decoder(
        z,
        training=training
    )

    return reconstruction

# =====
# TRAINING STEP
# =====

def train_step(self, data):

    # x_train is passed directly
    if isinstance(data, tuple):
        data = data[0]

    with tf.GradientTape() as tape:

        # -----
        # ENCODER
        # -----

        z_mean, z_log_var, z = self.encoder(
            data,
            training=True
        )

        # -----
        # DECODER
        # -----

        reconstruction = self.decoder(
            z,
            training=True
        )

        # -----
        # RECONSTRUCTION LOSS
        # -----

```

```

reconstruction_loss = tf.reduce_mean(
    tf.reduce_sum(
        tf.square(
            data - reconstruction
        ),
        axis=(1, 2, 3)
    )
)

# -----
# KL DIVERGENCE
# -----

kl_loss = -0.5 * tf.reduce_mean(
    tf.reduce_sum(
        1
        + z_log_var
        - tf.square(z_mean)
        - tf.exp(z_log_var),
        axis=1
    )
)

# -----
# TOTAL LOSS
# -----

total_loss = (
    reconstruction_loss
    + kl_loss
)

# =====
# BACKPROPAGATION
# =====

grads = tape.gradient(
    total_loss,
    self.trainable_weights
)

self.optimizer.apply_gradients(
    zip(
        grads,
        self.trainable_weights
    )
)

```

```

# =====
# UPDATE METRICS
# =====

self.total_loss_tracker.update_state(
    total_loss
)

self.reconstruction_loss_tracker.update_state(
    reconstruction_loss
)

self.kl_loss_tracker.update_state(
    kl_loss
)

return {
    "loss":
        self.total_loss_tracker.result(),

    "reconstruction_loss":
        self.reconstruction_loss_tracker.result(),

    "kl_loss":
        self.kl_loss_tracker.result()
}

# =====
# VALIDATION STEP
# =====

def test_step(self, data):

    if isinstance(data, tuple):
        data = data[0]

    # -----
    # ENCODER
    # -----

    z_mean, z_log_var, z = self.encoder(
        data,
        training=False
    )

```

```

# -----
# DECODER
# -----

reconstruction = self.decoder(
    z,
    training=False
)

# -----
# RECONSTRUCTION LOSS
# -----

reconstruction_loss = tf.reduce_mean(
    tf.reduce_sum(
        tf.square(
            data - reconstruction
        ),
        axis=(1, 2, 3)
    )
)

# -----
# KL DIVERGENCE
# -----

kl_loss = -0.5 * tf.reduce_mean(
    tf.reduce_sum(
        1
        + z_log_var
        - tf.square(z_mean)
        - tf.exp(z_log_var),
        axis=1
    )
)

# -----
# TOTAL LOSS
# -----

total_loss = (
    reconstruction_loss
    + kl_loss
)

# -----

```

```

# UPDATE METRICS
# -----

self.total_loss_tracker.update_state(
    total_loss
)

self.reconstruction_loss_tracker.update_state(
    reconstruction_loss
)

self.kl_loss_tracker.update_state(
    kl_loss
)

return {
    "loss":
        self.total_loss_tracker.result(),

    "reconstruction_loss":
        self.reconstruction_loss_tracker.result(),

    "kl_loss":
        self.kl_loss_tracker.result()
}

# =====
# 11. CREATE VAE MODEL
# =====

vae = VAE(
    encoder,
    decoder,
    name="fashion_mnist_vae"
)

# =====
# 12. COMPILE
# =====

vae.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001
    )
)

# =====

```

```

# 13. TRAIN
# =====

print("\n=====")
print("TRAINING VAE")
print("=====")

history = vae.fit(
    x_train,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test, None)
)

# =====
# 14. FINAL RESULTS
# =====

print("\n=====")
print("FINAL RESULTS")
print("=====")

print(
    "Final Training Total Loss:",
    history.history["loss"][-1]
)

print(
    "Final Training Reconstruction Loss:",
    history.history["reconstruction_loss"][-1]
)

print(
    "Final Training KL Loss:",
    history.history["kl_loss"][-1]
)

print(
    "Final Validation Total Loss:",
    history.history["val_loss"][-1]
)

print(
    "Final Validation Reconstruction Loss:",
    history.history["val_reconstruction_loss"][-1]
)

print(
    "Final Validation KL Loss:",

```

```

    history.history["val_kl_loss"][-1]
)

# =====
# 15. ENCODE TEST IMAGES
# =====

z_mean_test, z_log_var_test, z_test = encoder.predict(
    x_test,
    batch_size=256
)

# =====
# 16. RECONSTRUCT TEST IMAGES
# =====

reconstructed_images = decoder.predict(
    z_test,
    batch_size=256
)

# =====
# 17. ORIGINAL VS RECONSTRUCTED
# =====

n = 10

plt.figure(figsize=(20, 4))

for i in range(n):

    # -----
    # Original
    # -----

    ax = plt.subplot(
        2,
        n,
        i + 1
    )

    plt.imshow(
        x_test[i].squeeze(),
        cmap="gray"
    )

    plt.title("Original")
    plt.axis("off")

```

```

# -----
# Reconstructed
# -----

ax = plt.subplot(
    2,
    n,
    i + 1 + n
)

plt.imshow(
    reconstructed_images[i].squeeze(),
    cmap="gray"
)

plt.title("Reconstructed")
plt.axis("off")

plt.suptitle(
    "VAE: Original vs Reconstructed Images"
)

plt.show()

# =====
# 18. PLOT TOTAL LOSS
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Total Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Total Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "VAE Total Loss"
)

```

```

plt.legend()
plt.grid()

plt.show()

# =====
# 19. PLOT RECONSTRUCTION LOSS
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["reconstruction_loss"],
    label="Training Reconstruction Loss"
)

plt.plot(
    history.history["val_reconstruction_loss"],
    label="Validation Reconstruction Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "VAE Reconstruction Loss"
)

plt.legend()
plt.grid()

plt.show()

# =====
# 20. PLOT KL DIVERGENCE
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["kl_loss"],
    label="Training KL Loss"
)

plt.plot(
    history.history["val_kl_loss"],
    label="Validation KL Loss"
)

```

```

plt.xlabel("Epoch")
plt.ylabel("KL Divergence")

plt.title(
    "VAE KL Divergence"
)

plt.legend()
plt.grid()

plt.show()

# =====
# 21. VISUALIZE 2-D LATENT SPACE
# =====

plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    z_mean_test[:, 0],
    z_mean_test[:, 1],
    c=y_test,
    cmap="tab10",
    s=3,
    alpha=0.5
)

plt.colorbar(
    scatter,
    ticks=range(10)
)

plt.xlabel("Latent Dimension 1")
plt.ylabel("Latent Dimension 2")

plt.title(
    "VAE 2-D Latent Space"
)

plt.grid()

plt.show()

# =====
# 22. GENERATE NEW IMAGES
# =====

# Random samples from standard normal distribution

```

```

random_latent_vectors = np.random.normal(
    size=(10, latent_dim)
).astype("float32")

# Decode random latent vectors

generated_images = decoder.predict(
    random_latent_vectors
)

# =====
# 23. DISPLAY GENERATED IMAGES
# =====

plt.figure(figsize=(20, 3))
for i in range(10):
    ax = plt.subplot(
        1,
        10,
        i + 1
    )

    plt.imshow(
        generated_images[i].squeeze(),
        cmap="gray"
    )

    plt.title("Generated")
    plt.axis("off")

plt.suptitle(
    "VAE: Newly Generated Fashion Images"
)

plt.show()

# =====
# 24. DISPLAY RANDOM LATENT POINTS
# =====

print("\nRandom latent vectors used for generation:")
print(random_latent_vectors)

```

Original Training Shape: (60000, 28, 28)
Original Testing Shape : (10000, 28, 28)
Training Shape: (60000, 28, 28, 1)
Testing Shape : (10000, 28, 28, 1)

=====
ENCODER SUMMARY
=====

Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 28, 28, 1)	0	-
flatten_1 (Flatten) encoder_input[0]...	(None, 784)	0	
dense_5 (Dense) [0]	(None, 256)	200,960	flatten_1[0]
dense_6 (Dense)	(None, 128)	32,896	dense_5[0][0]
z_mean (Dense)	(None, 2)	258	dense_6[0][0]
z_log_var (Dense)	(None, 2)	258	dense_6[0][0]
sampling (Sampling) [0]	(None, 2)	0	z_mean[0][0], z_log_var[0]

Total params: 234,372 (915.52 KB)

Trainable params: 234,372 (915.52 KB)

Non-trainable params: 0 (0.00 B)

=====

DECODER SUMMARY

=====

Model: "decoder"

Layer (type)	Output Shape
Param #	
decoder_input (InputLayer)	(None, 2)
0	
dense_7 (Dense)	(None, 128)
384	
dense_8 (Dense)	(None, 256)
33,024	
dense_9 (Dense)	(None, 784)
201,488	
reshape_1 (Reshape)	(None, 28, 28, 1)
0	

Total params: 234,896 (917.56 KB)

Trainable params: 234,896 (917.56 KB)

Non-trainable params: 0 (0.00 B)

=====

TRAINING VAE

=====

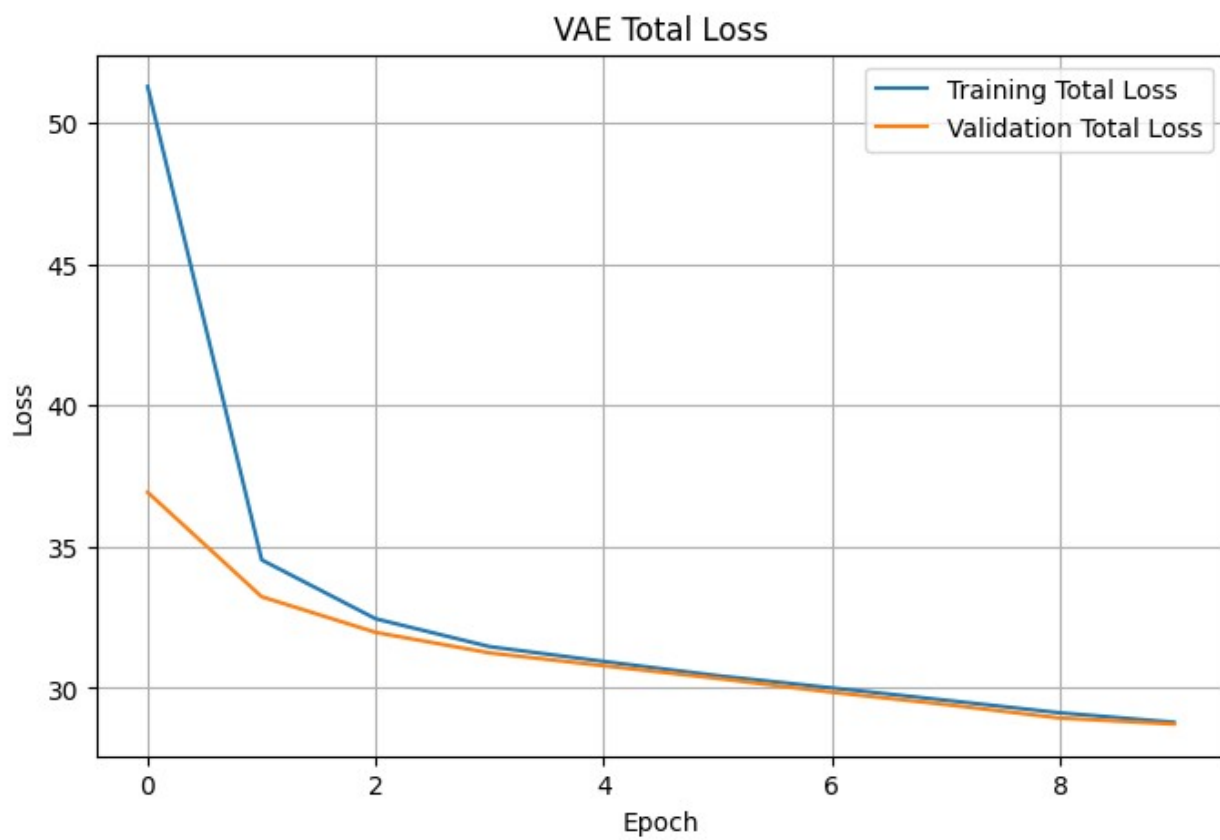
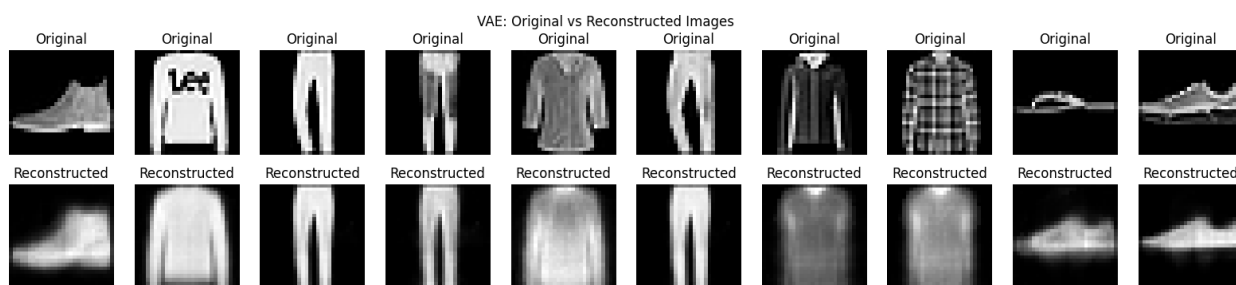
Epoch 1/10

235/235 ————— 6s 14ms/step - kl_loss: 3.5904 - loss:
51.2747 - reconstruction_loss: 47.6843 - val_kl_loss: 3.9128 -
val_loss: 36.9270 - val_reconstruction_loss: 33.0142
Epoch 2/10
235/235 ————— 5s 13ms/step - kl_loss: 4.1437 - loss:
34.5360 - reconstruction_loss: 30.3923 - val_kl_loss: 4.3465 -
val_loss: 33.2322 - val_reconstruction_loss: 28.8858
Epoch 3/10
235/235 ————— 3s 15ms/step - kl_loss: 4.3201 - loss:
32.4519 - reconstruction_loss: 28.1318 - val_kl_loss: 4.2996 -
val_loss: 31.9703 - val_reconstruction_loss: 27.6708
Epoch 4/10
235/235 ————— 5s 13ms/step - kl_loss: 4.3786 - loss:
31.4661 - reconstruction_loss: 27.0876 - val_kl_loss: 4.3530 -
val_loss: 31.2450 - val_reconstruction_loss: 26.8919
Epoch 5/10
235/235 ————— 3s 14ms/step - kl_loss: 4.4262 - loss:
30.9433 - reconstruction_loss: 26.5170 - val_kl_loss: 4.4556 -
val_loss: 30.7922 - val_reconstruction_loss: 26.3366
Epoch 6/10
235/235 ————— 4s 15ms/step - kl_loss: 4.4685 - loss:
30.4385 - reconstruction_loss: 25.9701 - val_kl_loss: 4.5433 -
val_loss: 30.3424 - val_reconstruction_loss: 25.7992
Epoch 7/10
235/235 ————— 5s 13ms/step - kl_loss: 4.5244 - loss:
30.0075 - reconstruction_loss: 25.4831 - val_kl_loss: 4.7528 -
val_loss: 29.8551 - val_reconstruction_loss: 25.1023
Epoch 8/10
235/235 ————— 6s 16ms/step - kl_loss: 4.5841 - loss:
29.5710 - reconstruction_loss: 24.9870 - val_kl_loss: 4.6213 -
val_loss: 29.4209 - val_reconstruction_loss: 24.7996
Epoch 9/10
235/235 ————— 3s 13ms/step - kl_loss: 4.6515 - loss:
29.1263 - reconstruction_loss: 24.4749 - val_kl_loss: 4.4691 -
val_loss: 28.9434 - val_reconstruction_loss: 24.4743
Epoch 10/10
235/235 ————— 3s 13ms/step - kl_loss: 4.6917 - loss:
28.7927 - reconstruction_loss: 24.1010 - val_kl_loss: 4.5540 -
val_loss: 28.7280 - val_reconstruction_loss: 24.1739

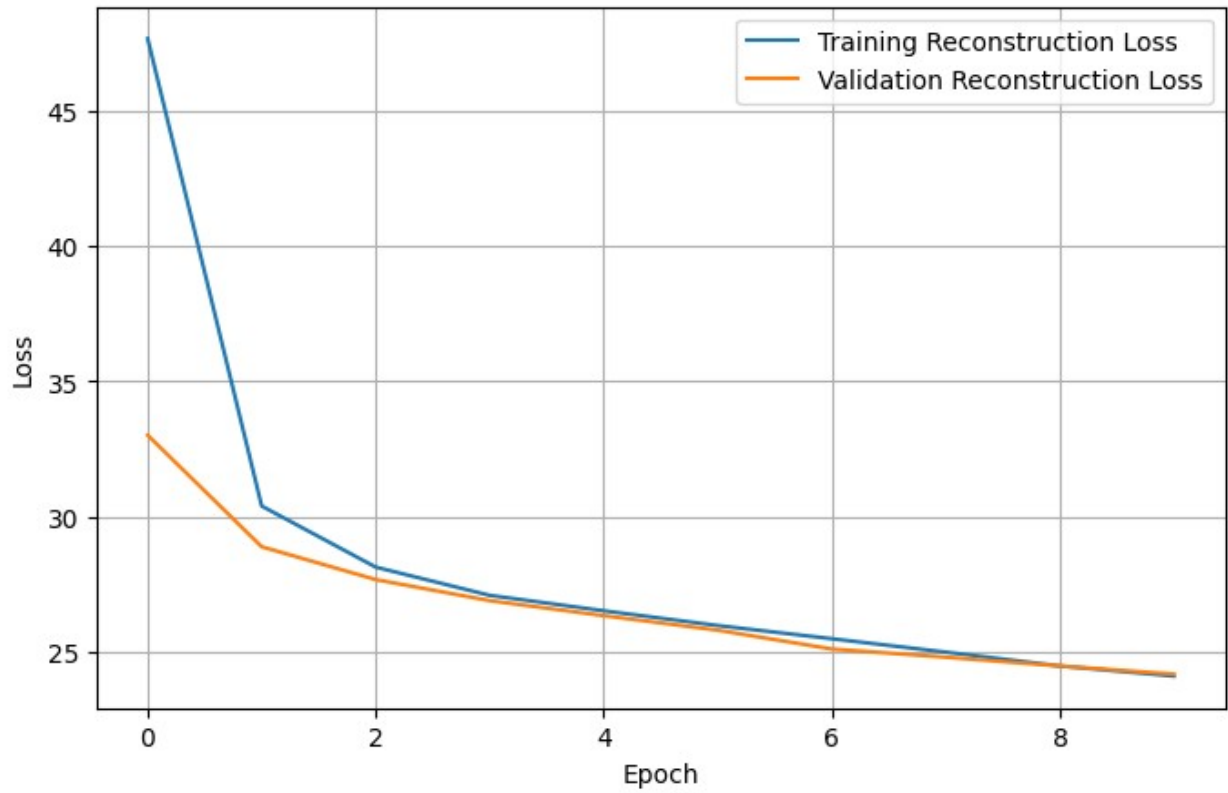
=====
FINAL RESULTS
=====

Final Training Total Loss: 28.792722702026367
Final Training Reconstruction Loss: 24.100982666015625
Final Training KL Loss: 4.691739559173584
Final Validation Total Loss: 28.72796058654785
Final Validation Reconstruction Loss: 24.17393684387207
Final Validation KL Loss: 4.5540266036987305

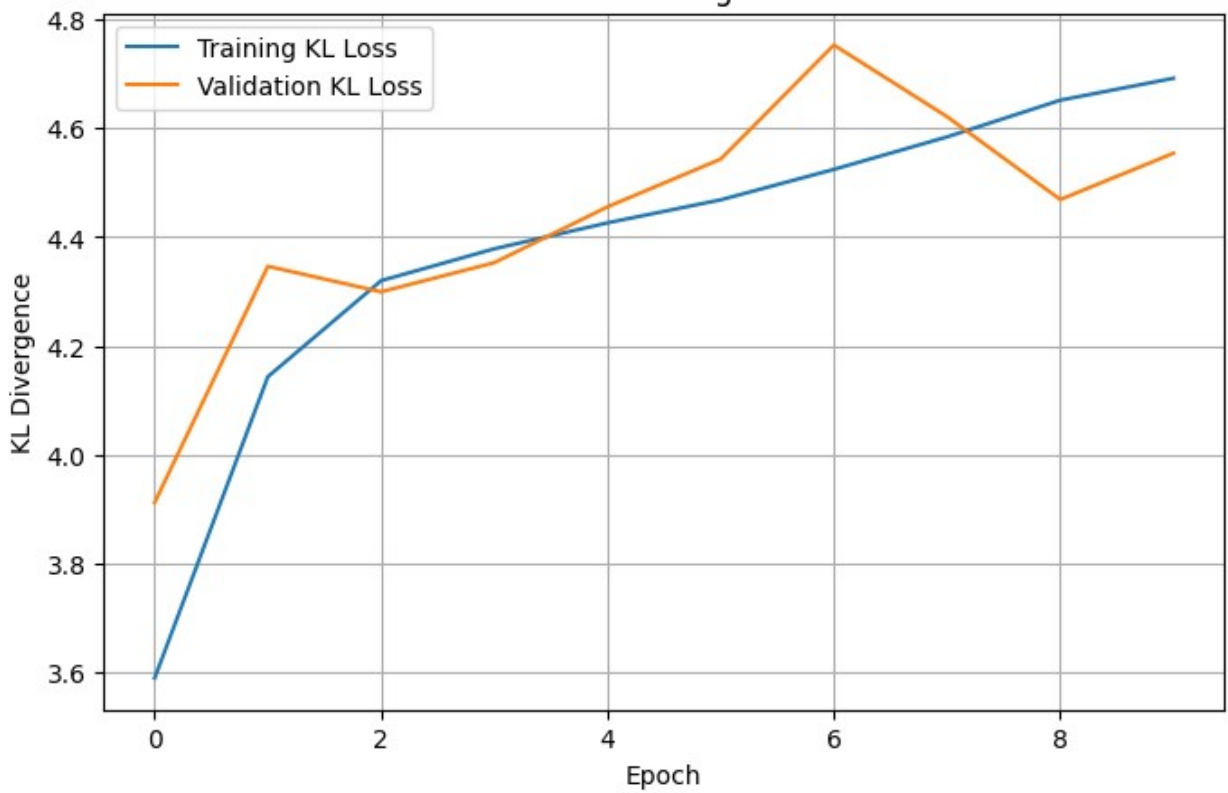
40/40 ————— 0s 4ms/step
40/40 ————— 0s 4ms/step

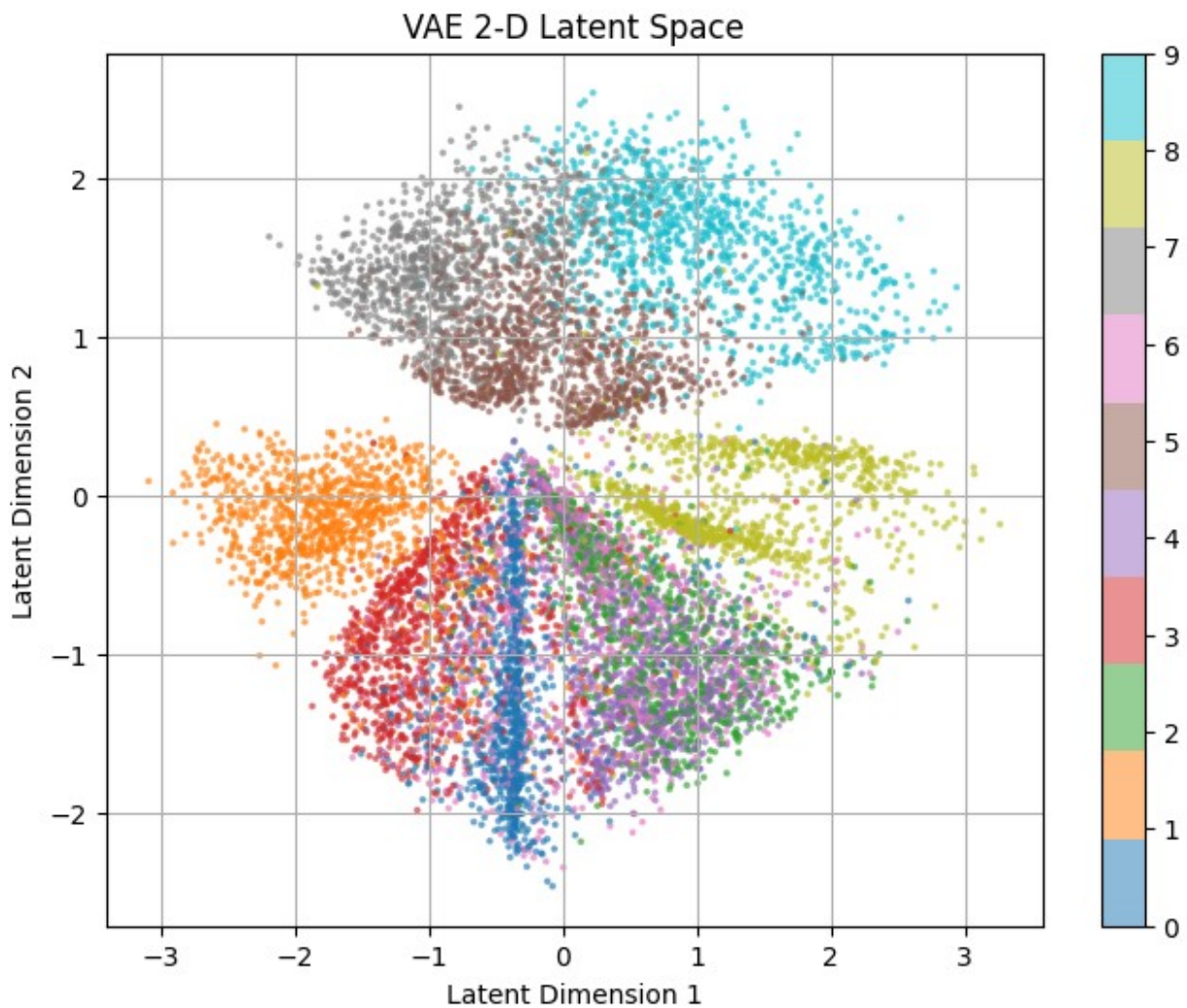


VAE Reconstruction Loss



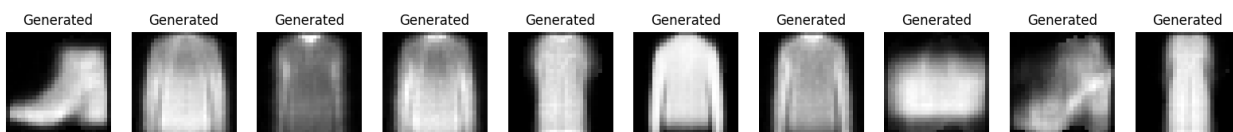
VAE KL Divergence





1/1 ————— 0s 39ms/step

VAE: Newly Generated Fashion Images



Random latent vectors used for generation:

```
[ [ 0.96730477  1.2348789 ]  
[ 0.9651722   -0.4998417 ]  
[ -0.14101122 -0.14339812 ]  
[ 0.5070085   -0.26051068 ]  
[ -0.67285854 -0.83781934 ]  
[ 0.8996439   -1.6717463  ]  
[ 0.46256208  -0.5076146  ]
```

```
[ 1.3770329  0.04089083]
[ 0.75456727 0.5203815 ]
[-1.1829195 -1.0622097 ]]
```

Original → Reconstructed

The reconstructed images preserve the basic category/shape, but they are noticeably smoother and blurrier.

For example:

shoe → still looks like a shoe

trousers → still looks like trousers

shirt → still looks like a shirt

checked shirt → general shirt shape remains, but the check pattern is largely lost

"The VAE has learned the important overall structure of the object, but because we forced the latent representation to only 2 dimensions, a lot of fine information has been discarded."

This plot is actually the main result of the VAE.

You have:

Latent Dimension 1 × Latent Dimension 2

and each point represents an image.

The colors represent the 10 Fashion-MNIST classes (0–9).

You can see that the classes are not completely random. Several groups occupy different regions, although there is significant overlap.

What does that mean?

"The VAE has converted a 784-dimensional image into just two numbers, yet those two numbers still contain information about what kind of object the image represents."

That's quite powerful.

Why are there clusters?

Images that have similar visual characteristics tend to be placed closer together in latent space.

Conceptually:

Similar images



Similar latent representation



Close points in latent space

VAE result — generated images

The generated images are actually showing the main reason we use VAE.

we sampled:

$z \sim N(0,1)$

and gave those random latent points to the decoder.

The decoder produced images that look like:

shoes

shirts

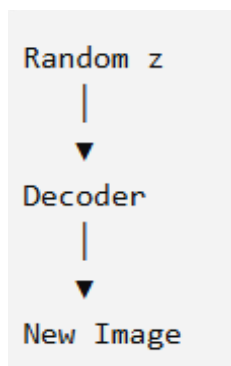
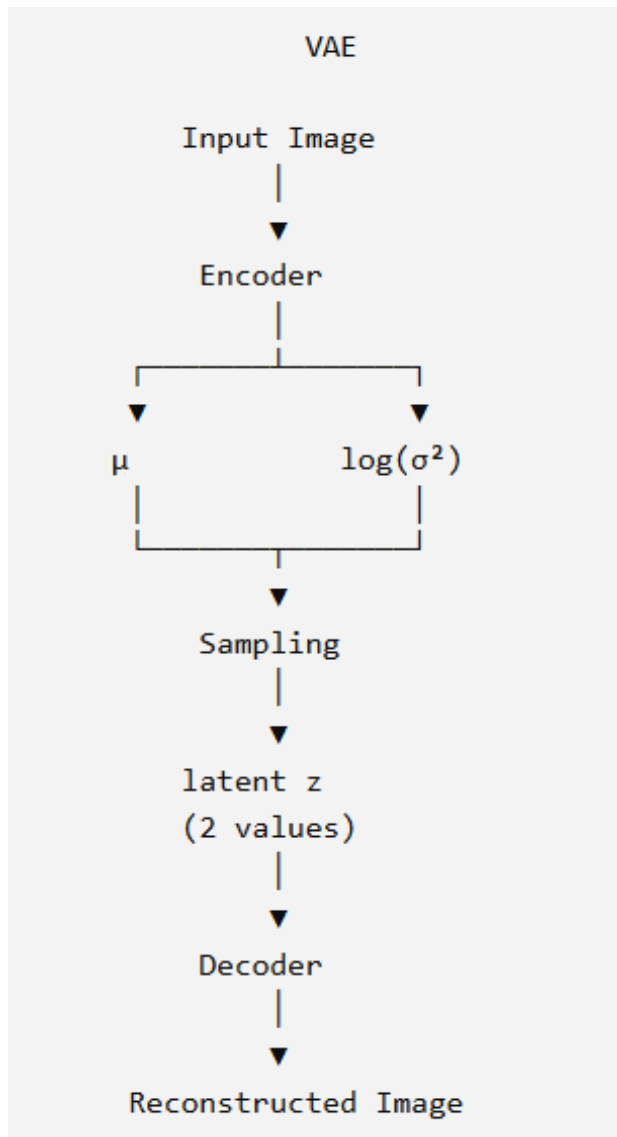
trousers

other Fashion-MNIST-like objects

They aren't copies of the original images.

"We didn't give the decoder an actual Fashion-MNIST image. We gave it a random point from the latent distribution, and it generated an image."

That's what makes VAE different from your ordinary autoencoder.



what does KL loss mean in your result?

KL loss ≈ 3.05 Reconstruction loss ≈ 65.51 Total loss ≈ 68.56

The relationship is:

Total loss = Reconstruction loss + KL divergence

So:

$$68.56 \approx 65.51 + 3.05$$

Reconstruction loss asks:

"Did I reconstruct the image correctly?"

KL divergence asks:

"Is my learned latent distribution behaving like the desired standard normal distribution?"

So VAE has two competing goals:

Good reconstruction + Well-structured latent space $\downarrow$ VAE

	Exercise 1	Exercise 2	Exercise 3
Model	Fully Connected AE	Convolutional AE	VAE
Main purpose	Reconstruction	Image reconstruction	Reconstruction + generative modeling
Input	784	28×28×1	28×28×1
Latent representation	Fixed vector	Feature map	Probability distribution
Latent example	32 values	7×7×64	$\mu, \sigma \rightarrow z$
Spatial information	Limited	Strong	Limited in this simple VAE
Sampling	No	No	Yes
Generate new images	No	No	Yes
Reconstruction	Usually blurry	Usually better spatial structure	Blurry here because latent_dim=2
Main concept	Compression	Spatial feature learning	Probabilistic latent space

Exercise 1 — Fully Connected AE

Can we compress an image and reconstruct it?

$$784 \rightarrow 32 \rightarrow 784$$

result: Compression causes information loss.

Exercise 2 — Convolutional AE

Can we reconstruct images better if we preserve spatial structure?

Image → Conv → latent feature maps → Conv → Image

result: CNNs are better suited to spatial image features.

Exercise 3 — VAE

"Can we do something more than reconstruction? Can we learn a meaningful latent space and generate new images?"

Image

↓

Probability distribution

↓

latent z

↓

Decoder

↓

Image

And then:

Random z

↓

Decoder

↓

NEW IMAGE

result: The latent space itself becomes useful and can be sampled.

#"The first autoencoder learns to compress, the convolutional autoencoder learns to compress while preserving spatial structure, and the VAE learns a structured latent distribution that allows us not only to reconstruct images but also to generate new ones."

Exploration & Interpretation — 5 Questions

Q1. AE vs VAE latent representation

Compare the latent representation of:

Fully Connected AE VAE

Question:

How is the latent representation of a VAE different from the fixed bottleneck representation of a conventional autoencoder?

Q2. What happens when latent dimension changes?

Try:

latent_dim = 2 latent_dim = 8 latent_dim = 16

Observe reconstruction quality.

Interpret:

How does increasing latent dimensionality affect reconstruction quality and the amount of information retained?

Q3. What does KL divergence contribute?

Observe the:

reconstruction loss KL loss total loss

Question:

Why does the VAE need KL divergence in addition to reconstruction loss?

Expected direction:

Reconstruction loss → reconstruct the image well.

KL divergence → keep the learned latent distribution close to a standard normal distribution.

Q4. Can the VAE generate new images?

Randomly sample latent vectors and generate images.

Question:

Are the generated images exact copies of training images, or do they represent new variations?
What does this tell you about the VAE latent space?

Q5. Why can a VAE generate images while a basic AE is mainly used for reconstruction?

Students should explain the role of:

probability distribution sampling structured latent space